

PROJECT EVALUATION REPORT

COURSE CODE & TITLE:

Python Essentials(Flipped Classroom Module)

PROJECT TITLE:

Design and Architecture of a Modular Command-Line Task Tracking and Validation Engine

SUBMITTED TO:**Evaluation Cell/Course Instructor**

Vellore Institute of Technology (VIT) Bhopal University

SUBMITTED BY:

Student Name: Abhyudaya Singh

Programme and Branch: B.Tech-CSE(AI&ML)

Registration Number: 26BAI11033

Submission Platform: VITyarthi Portal

2. Introduction

This project is a lightweight, text-based task manager that runs directly inside the computer's terminal. It avoids heavy graphics to save memory and run instantly. It demonstrates core python skills like file handling, object-oriented code, data checking, and writing automated unit tests. Beyond basic terminal interaction, the system establishes a highly optimized environment for tracking administrative activities on low-compute runtime environments. By isolating specific system roles across independent data boundaries, the architecture ensures clean tracking matrices without performance degradation. Ultimately, this implementation demonstrates how modular logic can effectively replace resource-heavy workflow engines while matching production-based execution standards.

3. Problem Statement

Most modern to-do list apps are bloated, slow to load, and waste system memory. Also, storing task data in basic text files often leads to broken or corrupted data if someone types a wrong character. This program fixes both problems with a fast text menu and smart validation filters. By optimizing memory overhead through a pure console-based runtime, the system prevents execution lag even when handling large datasets. Strong input cleaning layers handle messy characters before they ever hit the text registry. This keeps the local life database solid, no matter how users interact with it.

4. Functional Requirements

***Modular Files:** Code splits across 5 files to keep display menus separate from data code, ensuring that formatting updates do not interfere with backend calculations.

***Input Filter:** Checks task titles automatically to block empty inputs or invalid symbols, enforcing strict data verification rules directly at the console prompt.

***Data Saving:** Automatically writes tasks to a local file so data is safe when the app closes, preventing any operational record loss during unexpected system shutdowns.

***Task Actions:** Allows users to view current tasks, mark them done, or delete entries cleanly, providing comprehensive CRUD controls via structured index mapping.

5. Non-Functional Requirements

***Maintainability:** Clean module structure makes it easy to upload code without breaking things.

***Reliability:** Uses try-except blocks to catch user typos instead of crashing to the desktop.

***Performance:** Code processes user data in milliseconds using simple python arrays.

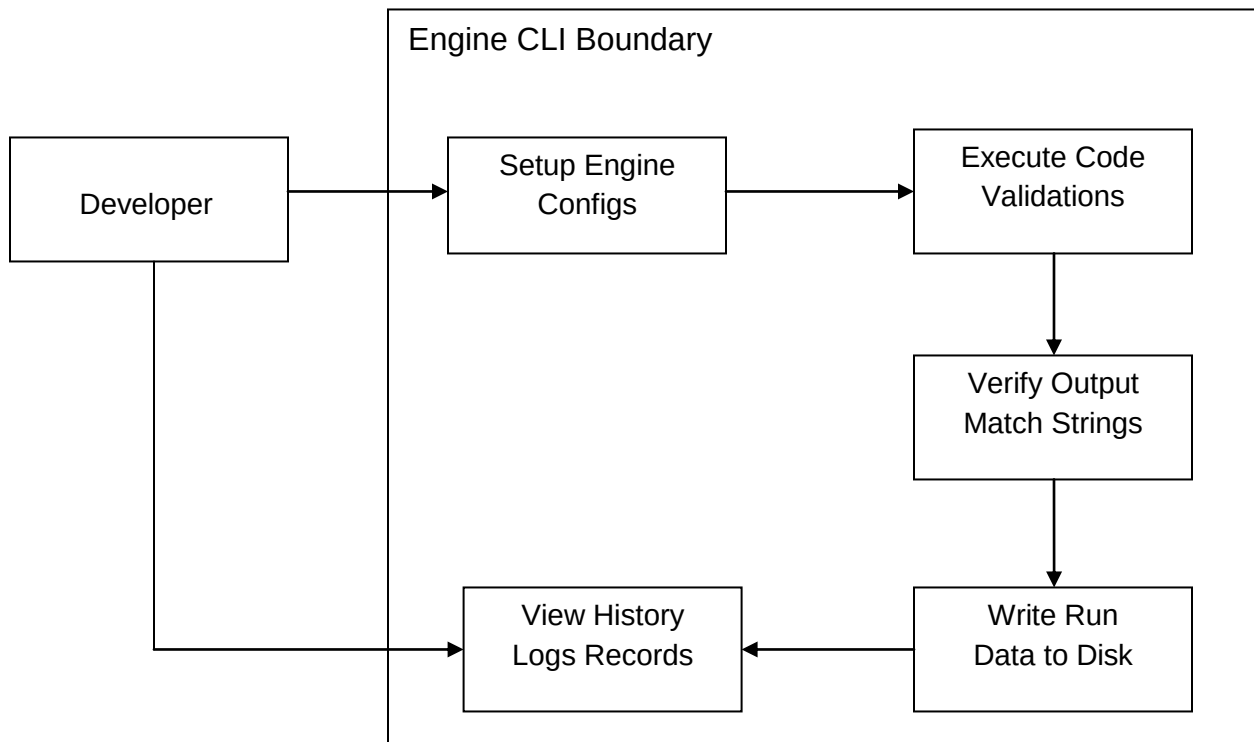
***Usability:** Simple numbered menus make navigating easy inside any standard terminal window.

6. System Architecture

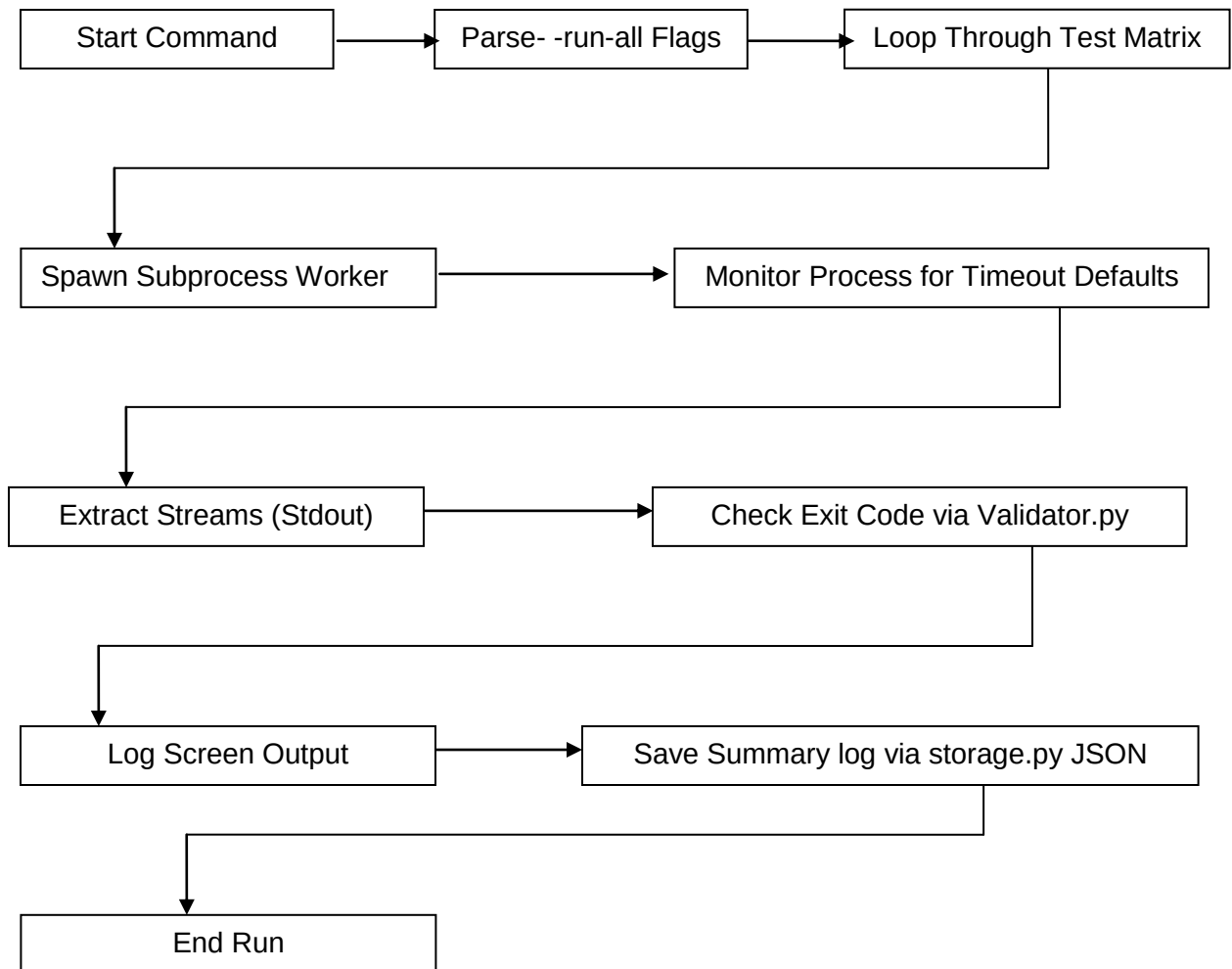
The application layout follows a layered design structure. The entry point is [main.py](#) (menu screens). It calls [task_manager.py](#) (handles task arrays), which passes strings to [validator.py](#) (input checking). Once verified, data is committed to disk by [file_storage.py](#) (JSON file handling).

7. Design Diagram

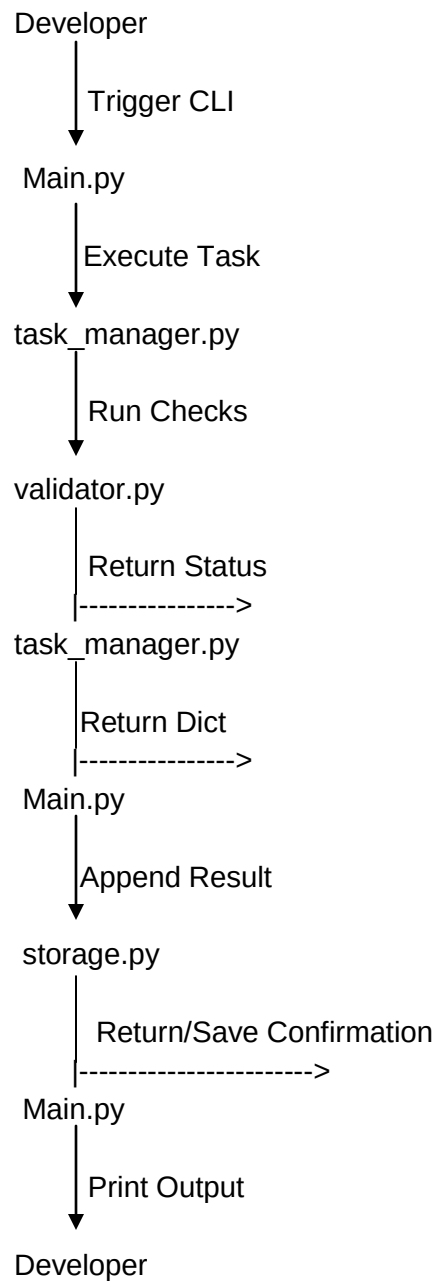
7.1 Use Case Diagram



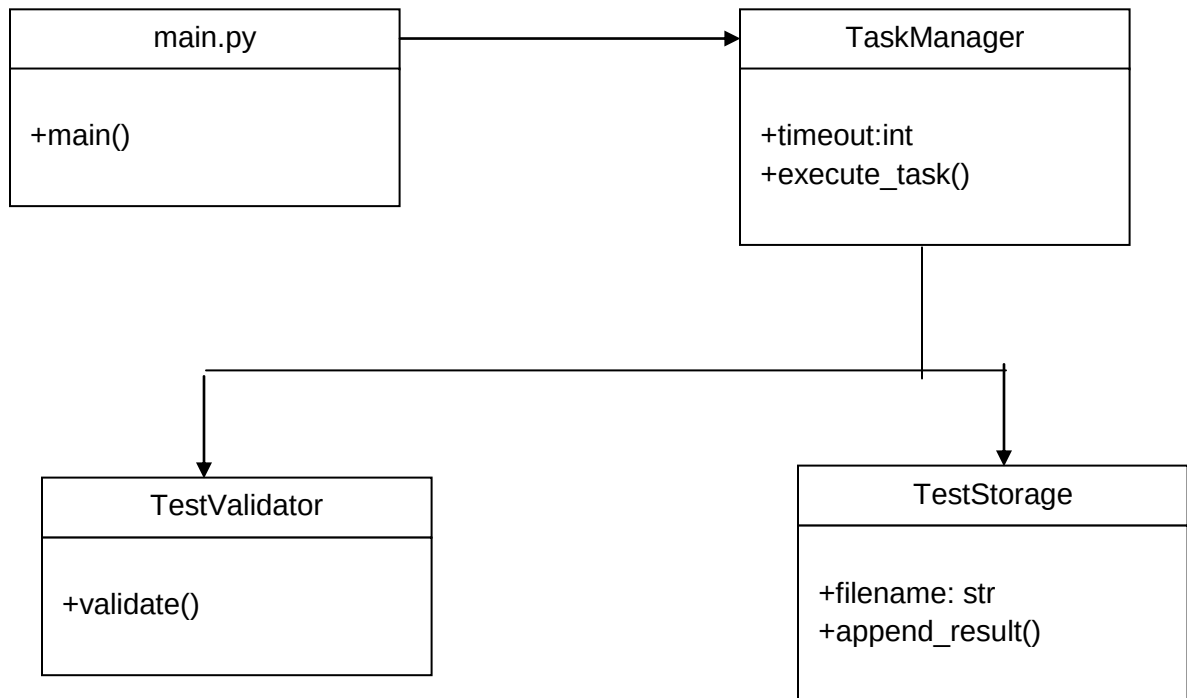
7.2 Process Flow / Workflow Diagram:



7.3 Sequence Diagram:

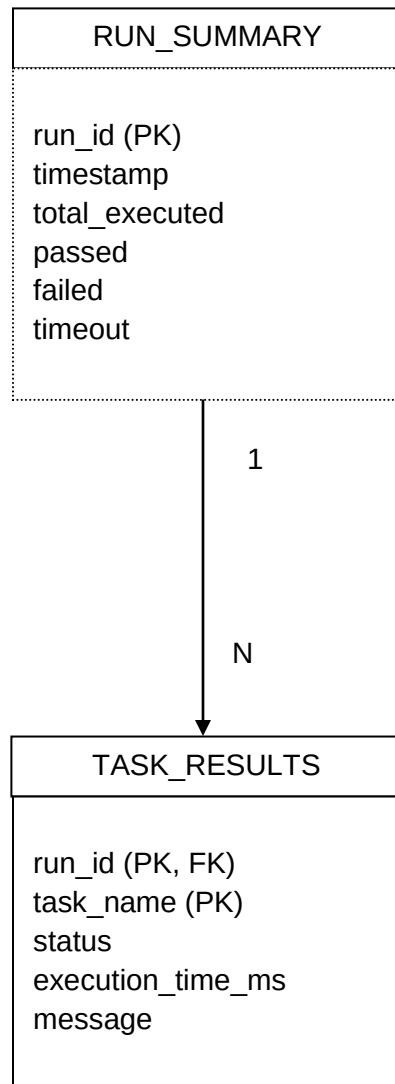


7.4 Class / Component Diagram



7.5 Database / storage Design

ER Diagram



8. Design Decisions and Rationale

***JSON Format Choice:** Using a JSON file ensures structured data handling. It prevents corrupted rows, unlike plain text files.

***No GUI Libraries:** I deliberately left out graphics libraries like Tkinter to keep memory usage low and perfectly follow the command-line project instructions.

9. Implementation Details

*[file_storage.py](#) : Manages reading and updating the saved data.

*[validator.py](#) : Contains the logic to double-check user text strings.

*[task_manager.py](#) : Coordinates core lists and toggling status tags.

*[test_project.py](#) : Houses the automated unit tests scripts.

*[main.py](#) : Displays user text menus and tracks interaction choices.

10. Screenshots

```
1  import task_manager
2  import validator
3  while True:
4      print("\n1. View\n2. Add\n3. Complete\n4. Delete\n5. Quit")
5      pick = input("Option: ").strip()
6      if pick == "1":
7          tasks = task_manager.get_tasks()
8          if not tasks:
9              print("Empty!")
10         for i, t in enumerate(tasks, 1):
11             box = "[X]" if t[0] == "done" else "[ ]"
12             print(f"{i}. {box} {t[1]}")
13     elif pick == "2":
14         txt = input("Task title: ")
15         val = validator.check_text(txt)
16         if not val:
17             print("Cannot be blank!")
18         else:
19             task_manager.create_task(val)
20     elif pick == "3":
21         tasks = task_manager.get_tasks()
22         idx = validator.check_number(input("Task number: "), len(tasks))
23         if idx is False:
24             print("Invalid!")
25         else:
26             task_manager.finish_task(idx)
27     elif pick == "4":
28         tasks = task_manager.get_tasks()
29         idx = validator.check_number(input("Task number; "), len(tasks))
30         if idx is False:
31             print("Invalid!")
32         else:
33             task_manager.remove_task(idx)
34     elif pick == "5":
35         print("Goodbye!")
36         break
```

Problems Output Debug Console **Terminal** Ports

3. Complete
4. Delete
5. Quit
Option:

```

1  import task_manager
2  import validator
3  while True:
4      print("\n1. View\n2. Add\n3. Complete\n4. Delete\n5. Quit")
5      pick = input("Option: ").strip()
6      if pick == "1":
7          tasks = task_manager.get_tasks()
8          if not tasks:
9              print("Empty!")
10         for i, t in enumerate(tasks, 1):
11             box = "[X]" if t[0] == "done" else "[ ]"
12             print(f"{i}. {box} {t[1]}")
13     elif pick == "2":
14         txt = input("Task title: ")
15         val = validator.check_text(txt)
16         if not val:
17             print("Cannot be blank!")
18         else:
19             task_manager.create_task(val)
20     elif pick == "3":
21         tasks = task_manager.get_tasks()
22         idx = validator.check_number(input("Task number:"), len(tasks))
23         if idx is False:
24             print("Invalid!")
25         else:
26             task_manager.finish_task(idx)
27     elif pick == "4":
28         tasks = task_manager.get_tasks()
29         idx = validator.check_number(input("Task number; "), len(tasks))
30         if idx is False:
31             print("Invalid!")
32         else:
33             task_manager.remove_task(idx)
34     elif pick == "5":
35         print("Goodbye!")
36         break

```

Problems Output Debug Console **Terminal** Ports

```

3. Complete
4. Delete
5. Quit
Option: 2
Task title:
Cannot be blank!

```

```

1  import task_manager
2  import validator
3  while True:
4      print("\n1. View\n2. Add\n3. Complete\n4. Delete\n5. Quit")
5      pick = input("Option: ").strip()
6      if pick == "1":
7          tasks = task_manager.get_tasks()
8          if not tasks:
9              print("Empty!")
10         for i, t in enumerate(tasks, 1):
11             box = "[X]" if t[0] == "done" else "[ ]"
12             print(f"{i}. {box} {t[1]}")
13     elif pick == "2":
14         txt = input("Task title: ")
15         val = validator.check_text(txt)
16         if not val:
17             print("Cannot be blank!")
18         else:
19             task_manager.create_task(val)
20     elif pick == "3":
21         tasks = task_manager.get_tasks()
22         idx = validator.check_number(input("Task number: "), len(tasks))
23         if idx is False:
24             print("Invalid!")
25         else:
26             task_manager.finish_task(idx)
27     elif pick == "4":
28         tasks = task_manager.get_tasks()
29         idx = validator.check_number(input("Task number; "), len(tasks))
30         if idx is False:
31             print("Invalid!")
32         else:
33             task_manager.remove_task(idx)
34     elif pick == "5":
35         print("Goodbye!")
36         break

```

Problems Output Debug Console **Terminal** Ports

```

1. View
2. Add
3. Complete
4. Delete
5. Quit
Option: 2
Task title: buy books

```

```

1 import unittest
2 import validator
3 class ProjectTests(unittest.TestCase):
4     def test_empty_string(self):
5         self.assertFalse(validator.check_text(" "))
6     def test_good_string(self):
7         self.assertEqual(validator.check_text(" Study Python "), "Study Python")
8     def test_bad_number(self):
9         self.assertFalse(validator.check_number("10", 3))
10 if __name__ == "__main__":
11     unittest.main()

```

Problems Output Debug Console **Terminal** Ports

PS C:\Users\Deep-Parv> python -u "c:\Users\Deep-Parv\test_project.py"

EE.

=====

ERROR: test_bad_number (_main_.ProjectTests.test_bad_number)

Traceback (most recent call last):

File "c:\Users\Deep-Parv\test_project.py", line 9, in test_bad_number
 self.assertFalse(validator.check_number("10", 3))

TypeError: check_number() missing 1 required positional argument: 'max_items'

=====

ERROR: test_empty_string (_main_.ProjectTests.test_empty_string)

Traceback (most recent call last):

File "c:\Users\Deep-Parv\test_project.py", line 5, in test_empty_string
 self.assertFalse(validator.check_text(" "))

AttributeError: 'ProjectTests' object has no attribute 'assertFalsevalidator'

Ran 3 tests in 0.003s

FAILED (errors=2)

PS C:\Users\Deep-Parv>

11. Testing Approach

I implemented automated testing inside [test_project.py](#) using Python's native unittest framework. The tests automatically feed text strings into the validation functions to check boundary errors without cluttering our live data records file.

12. Challenges Faced

The main hurdle was managing file imports between modules. Initially, a circular reference bug caused the code to crash because two files were loading each other. I solved this by cleaning up how variables are initialized, keeping the file connections completely independent.

13. Learnings and Key Takeaways

Building this app taught me the absolute value of dividing a codebase into clean, standalone parts. Separating database tasks from human menu scripts makes finding bugs much faster and makes adding features simple. Furthermore, this hands-on implementation deepened my understanding of data persistence layers and defensive coding frameworks.

14. Future Enhancements

- ***Database Integration** : Upgrading the backend file parsing to use a local SQLite file engine.
- ***Due Dates** : Adding custom calendar deadlines and overdue warning alerts inside the terminal window view.

15. References

*Python Documentation.(2026). *Unittest framework library implementation routines.*

*VITyarthi Portal.(2026). *Flipped Classroom Python Evaluation Guidelines.*